

CEN3024C Module 8 Project

For this project we will enhance the TaskManager application by implementing automated testing and continuous integration (CI) using GitHub Actions. Building on previous work, we will create and verify unit tests, then automate the build and test process through a workflow file in the repository.

Before starting, note the change made to the existing `src/main/resources/application.properties` file made for the test environment:

```
spring.config.activate.on-profile=!test
```

This ensures that the main configuration (including database setup and `data.sql` execution) is excluded when the Spring “test” profile is active. Test classes can then safely run against an in-memory H2 database or mocked repositories without overwriting or connecting to the real PostgreSQL instance

The `test/application.properties` file has also been added.

```
spring.profiles.active=test  
spring.sql.init.mode=never  
spring.datasource.driver-class-name=org.h2.Driver  
spring.datasource.username=sa  
spring.datasource.password=
```

```
spring.jpa.hibernate.ddl-auto=create-drop  
spring.jpa.show-sql=true  
spring.jpa.properties.hibernate.format_sql=true
```

```
# Optional, but helps for @DataJpaTest isolation  
spring.test.database.replace=ANY
```

This file configures an isolated in-memory H2 environment for testing, disabling `data.sql` execution and using `create-drop` to rebuild the schema for each test run. It also enables SQL logging and allows Spring to replace any existing data source to ensure repeatable test executions.

1. Create Task Tests:

- Use the `UserRepository` and `UserService` test examples in the GitHub Classroom repository as references.
- Implement new tests for the `TaskRepository` and `TaskService` classes.
- For `TaskRepository`: create both an in-memory test class and a mocking test class.
- For `TaskService`: create one mocking test class.

Sample code:

```
// TaskRepositoryInMemoryTest.java  
  
package edu.fscj.cen3024c.taskmanager.repositories;  
  
import edu.fscj.cen3024c.taskmanager.entities.Task;
```

```

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.orm.jpa.TestEntityManager;
import org.springframework.boot.test.autoconfigure.jdbc.AutoConfigureTestDatabase;
import org.springframework.test.context.TestPropertySource;

import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;

@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.ANY)
@TestPropertySource(properties = {
    "spring.sql.init.mode=never" // skip data.sql
})
public class TaskRepositoryInMemoryTest {

    @Autowired
    private TestEntityManager entityManager;

    @Autowired
    private TaskRepository taskRepository;

    @Test
    void saveTask_ShouldPersistTaskInDatabase() {
        Task task = new Task();
        task.setTitle("Complete Assignment");
        task.setStatus("Pending");
        task.setDueDate(java.time.LocalDate.now().plusDays(3).toString());

        Task savedTask = taskRepository.save(task);
        Task found = entityManager.find(Task.class, savedTask.getId());
        assertThat(found).isEqualTo(savedTask);
    }

    @Test
    void findById_ShouldReturnTask_WhenExists() {
        Task task = new Task();
        task.setTitle("Write Report");
        task.setStatus("In Progress");
        task.setDueDate(java.time.LocalDate.now().toString());
        entityManager.persistAndFlush(task);

        Optional<Task> found = taskRepository.findById(task.getId());
        assertThat(found).isPresent();
        assertThat(found.get().getTitle()).isEqualTo("Write Report");
    }
}

```

```

    @Test
    void findById_ShouldReturnEmpty_WhenNotExists() {
        assertThat(taskRepository.findById(999)).isEmpty();
    }
}

```

// TaskRepositoryMockTest.java

```

package edu.fscj.cen3024c.taskmanager.repositories;

import edu.fscj.cen3024c.taskmanager.entities.Task;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.junit.jupiter.MockitoExtension;
import org.mockito.Mock;

import java.util.Optional;

import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

@ExtendWith(MockitoExtension.class)
public class TaskRepositoryMockTest {

    @Mock
    private TaskRepository taskRepository;

    // Tests findById method
    @Test
    public void testFindById_ShouldReturnTask_WhenTaskExists() {
        // Given
        Integer taskId = 1;
        Task expectedTask = new Task();
        expectedTask.setId(taskId);
        expectedTask.setTitle("Complete Mock Testing");

        when(taskRepository.findById(taskId)).thenReturn(Optional.of(expectedTask));

        // When
        Optional<Task> actualTask = taskRepository.findById(taskId);

        // Then
        assertTrue(actualTask.isPresent(), "Task should be found");
        assertEquals(expectedTask.getId(), actualTask.get().getId(), "Task ID should match");
    }
}

```

```

        assertEquals(expectedTask.getTitle(), actualTask.get().getTitle(), "Task title should match");

        verify(taskRepository, times(1)).findById(taskId);
    }

    // Verifies findById returns empty Optional when task ID does not exist
    @Test
    public void testFindById_ShouldReturnEmpty_WhenTaskDoesNotExist() {
        // Given
        Integer nonExistentId = 999;
        when(taskRepository.findById(nonExistentId)).thenReturn(Optional.empty());

        // When
        Optional<Task> actualTask = taskRepository.findById(nonExistentId);

        // Then
        assertFalse(actualTask.isPresent(), "No task should be found with non-existent ID");

        verify(taskRepository, times(1)).findById(nonExistentId);
    }
}

```

// TaskServiceTest.java

```

package edu.fscj.cen3024c.taskmanager.services;

import edu.fscj.cen3024c.taskmanager.dto.TaskDTO;
import edu.fscj.cen3024c.taskmanager.entities.Task;
import edu.fscj.cen3024c.taskmanager.repositories.TaskRepository;
import edu.fscj.cen3024c.taskmanager.repositories.UserRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Arrays;
import java.util.List;
import java.util.Optional;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;
import static org.assertj.core.api.Assertions.*;

```

```

@ExtendWith(MockitoExtension.class)
public class TaskServiceTest {

    @Mock
    private TaskRepository taskRepository;

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private TaskService taskService;

    // Tests findAll()
    @Test
    void findAll_ShouldReturnTaskList() {
        // Given
        Task task1 = new Task();
        task1.setId(1);
        task1.setTitle("Task 1");

        Task task2 = new Task();
        task2.setId(2);
        task2.setTitle("Task 2");

        when(taskRepository.findAll()).thenReturn(Arrays.asList(task1, task2));

        // When
        List<TaskDTO> result = taskService.findAll();

        // Then
        assertNotNull(result).hasSize(2);
        assertEquals(result.get(0).getTitle(), "Task 1");
        verify(taskRepository, times(1)).findAll();
    }

    // Tests save(Task)
    @Test
    void save_ShouldPersistTaskEntity() {
        // Given
        Task taskToSave = new Task();
        taskToSave.setTitle("New Task");
        taskToSave.setStatus("Pending");
        taskToSave.setDueDate(java.time.LocalDate.now().plusDays(7).toString());
    }

```

```

Task savedTask = new Task();
savedTask.setId(1);
savedTask.setTitle(taskToSave.getTitle());
savedTask.setStatus(taskToSave.getStatus());
savedTask.setDueDate(taskToSave.getDueDate());

when(taskRepository.save(any(Task.class))).thenReturn(savedTask);

// When
Task result = taskService.save(taskToSave);

// Then
assertThat(result).isNotNull();
assertThat(result.getId()).isEqualTo(1);
assertThat(result.getTitle()).isEqualTo("New Task");
verify(taskRepository, times(1)).save(any(Task.class));
}

// Tests retrieval by ID returning TaskDTO
@Test
void findById_ShouldReturnTaskDTO_WhenTaskExists() {
    // Given
    Integer taskId = 1;
    Task task = new Task();
    task.setId(taskId);
    task.setTitle("Existing Task");

    when(taskRepository.findById(taskId)).thenReturn(Optional.of(task));

    // When
    TaskDTO result = taskService.findById(taskId);

    // Then
    assertThat(result).isNotNull();
    assertThat(result.getTitle()).isEqualTo("Existing Task");
    verify(taskRepository, times(1)).findById(taskId);
}

// Tests deleteById()
@Test
void deleteById_ShouldDeleteTask() {
    // Given
    Integer taskId = 1;
    when(taskRepository.existsById(taskId)).thenReturn(true);

```

```

// When
taskService.deleteById(taskId);

// Then
verify(taskRepository, times(1)).deleteById(taskId);
}

// Tests deleteById throws exception when task does not exist
@Test
void deleteById_ShouldThrow_WhenTaskDoesNotExist() {
    // Given
    Integer taskId = 99;
    when(taskRepository.existsById(taskId)).thenReturn(false);

    // Then
    assertThatThrownBy(() -> taskService.deleteById(taskId))
        .assertInstanceOf(RuntimeException.class)
        .hasMessageContaining(String.valueOf(taskId));
}
}

```

2. Verify Tests

- Right-click on the java subfolder under the test directory in IntelliJ and select Run all tests.
- Ensure all tests display green checkmarks indicating success.
- Take a snip of the test results for submission

3. Create a GitHub Actions Workflow

- Create a .github/workflows folder, then create a new file named build-and-test.yml.

Sample Workflow Content

name: Build and Run TaskManager Application

on:

push:

paths:

- '**/*.java'

pull_request:

paths:

- '**/*.java'

workflow_dispatch: # Enables manual trigger

jobs:

test:

runs-on: ubuntu-latest

steps:

- **uses: actions/checkout@v3**

- name: Set up JDK 17
uses: actions/setup-java@v3
with:
 java-version: '17'
 distribution: 'temurin'
- name: Run Maven Tests
run: mvn -f pom.xml test

build-and-run:

- needs: test
- runs-on: ubuntu-latest
- steps:
 - uses: actions/checkout@v3
 - name: Set up JDK 17
uses: actions/setup-java@v3
with:
 java-version: '17'
 distribution: 'temurin'
 - name: Build with Maven
run: mvn -f pom.xml clean package -DskipTests
 - name: Run Spring Boot Application
run: |
 java -jar target/*.jar &
 echo \$! > springboot.pid
 timeout-minutes: 5
 - name: Shutdown Spring Boot Application
run: kill \$(cat springboot.pid)

- The workflow should automatically build and test the application when a .java file is added or modified.

4. Workflow Verification

- Commit and push your .yml workflow file.
- Push a change (this can be as simple as adding or changing a comment in a .java file).
- Navigate to the **Actions** tab in GitHub to verify the automated build and test process.
- Demonstrate a successful test run (either automatic or manual) using a screenshot of the executed workflow

<> CodeIssuesPull requestsActionsProjectsSecurityInsightsSettings

Workflow run was successfully requested.

Actions

New workflow

All workflows

Build and Run TaskManager Applica...

Management

Caches

Build and Run TaskManager Application

buildandtest.yml

1 workflow run

EventStatusBranchActor

This workflow has a workflow_dispatch event trigger.

Run workflow

Filter workflow runs

- From the Actions tab you can view your workflow run summaries:

The screenshot shows the 'Build and Run TaskManager Application' workflow page. At the top, it says '3 workflow runs'. Below this, there's a blue banner stating 'This workflow has a workflow_dispatch event trigger.' with a 'Run workflow' button. The main list shows three runs:

- comment #2** (green checkmark): Build and Run TaskManager Application #3: Commit 2cc819f pushed by ProfSingletary. Status: Success. Duration: 1m 2s. Triggered 1 minute ago.
- Build and Run TaskManager Application** (green checkmark): Build and Run TaskManager Application #2: Manually run by ProfSingletary. Status: Success. Duration: 1m 12s. Triggered 12 minutes ago.
- Build and Run TaskManager Application** (red X): Build and Run TaskManager Application #1: Manually run by ProfSingletary. Status: Failed. Duration: 11s. Triggered 19 minutes ago.

- Select a specific run instance to drill down on the results (a green checkmark means success)

The screenshot shows the detailed view for 'Build and Run TaskManager Application #2'. It has a green checkmark and is labeled 'Manually triggered 9 minutes ago'. The status is 'Success', total duration is '1m 12s', and there are no artifacts. The workflow is triggered by 'workflow_dispatch'.

On the left, there's a sidebar with 'Summary' selected, and a list of jobs: 'test' (green checkmark) and 'build-and-run' (green checkmark). Below the jobs list, there are links for 'Run details', 'Usage', and 'Workflow file'.

The main area shows a job summary for 'buildandtest.yml' with two jobs: 'test' (33s) and 'build-and-run' (30s), both with green checkmarks.

- You can find your build and test results by selecting the applicable job button in the Action. The output is very lengthy, but if you scroll to the bottom of the test output here is what a successful test run looks like:

```

9095 [INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.206 s -- in
edu.fscj.cen3024c.taskmanager.repository.UserRepositoryInMemoryTest
9096 [INFO] Running edu.fscj.cen3024c.taskmanager.repository.TaskRepositoryMockTest
9097 [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.449 s -- in
edu.fscj.cen3024c.taskmanager.repository.TaskRepositoryMockTest
9098 [INFO] Running edu.fscj.cen3024c.taskmanager.repository.UserRepositoryMockTest
9099 [INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.167 s -- in
edu.fscj.cen3024c.taskmanager.repository.UserRepositoryMockTest
9100 [INFO] Running edu.fscj.cen3024c.taskmanager.service.UserServiceTest
9101 [INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.382 s -- in
edu.fscj.cen3024c.taskmanager.service.UserServiceTest
9102 [INFO] Running edu.fscj.cen3024c.taskmanager.service.TaskServiceTest
9103 [INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.042 s -- in
edu.fscj.cen3024c.taskmanager.service.TaskServiceTest
9104 [INFO]

```

Submission

Submit your project and screensnips showing your test results to the GitHub classroom repo.